

# Graph Structure Learning for Traffic Prediction

Mahmood Amintoosi<sup>1\*</sup>

<sup>1\*</sup>Division of Computer Science, Department of Applied Mathematics,  
Faculty of Mathematical Sciences, Ferdowsi University of Mashhad,  
Bahonar Av., Mashhad, 91775, Khorasan-e Razavi, IRAN,  
ORCID:0000-0001-9640-6475.

Corresponding author(s). E-mail(s): [m.amintoosi@um.ac.ir](mailto:m.amintoosi@um.ac.ir);

## Abstract

Accurate traffic forecasting remains challenging due to complex spatial and temporal dependencies. Existing graph-based methods typically rely on predefined adjacency matrices constructed from physical proximity, although physical connections may not adequately represent functional dependencies in traffic data. To address this limitation, we investigate a Graph Structure Learning (GSL) framework in which the adjacency matrix is learned directly from traffic observations rather than predefined from physical connections. We consider both contemporaneous and multi-lag learned graph structures and evaluate them with spatial and spatiotemporal forecasting models on the real-world SZ-Taxi and Los-loop datasets. The results show that learned graph structures can improve traffic forecasting, with the largest and most consistent gains obtained when lag-specific structures are consumed according to their temporal alignment. On Los-loop, the best learned multi-lag configuration reduces RMSE by 14.5% relative to the graph-free baseline at the first forecasting horizon and by up to 43.0% relative to the physical-graph baseline across the evaluated horizons. On SZ-Taxi, the learned multi-lag structure provides only limited gains, reflecting the much sparser structure recovered from the data. Additional experiments with matched-sparsity graphs show that the observed gains on Los-loop cannot be explained by sparsity alone. Overall, the results indicate that learning the graph structure from traffic data can provide useful predictive information, although its effectiveness depends on the dataset and on how the learned structure is incorporated into the forecasting model. The learned graphs are interpreted as statistical dependencies rather than causal relationships. The paper code is available at <https://github.com/mamintoosi/TGCN-GSL-PyTorch>.

**Keywords:** Traffic Forecasting, Graph Knowledge Discovery, Graph Structure Learning, Graph Convolutional Networks, T-GCN, Multi-Lag Graph Learning

# 1 Introduction

Recent advances in graph knowledge discovery have opened new possibilities for understanding complex urban systems, particularly in transportation networks. This paper applies the principles of graph knowledge discovery to traffic prediction, and examines whether learning the graph structure from traffic data improves forecasting relative to graphs that are predefined from physical connectivity. In recent years, traffic prediction has garnered significant attention in smart urban transportation systems. In navigation, ride-hailing, and route planning, accurate traffic prediction plays a crucial role in reducing congestion and increasing the efficiency of transportation networks. Traffic prediction involves analyzing traffic conditions, including flow and speed, extracting traffic patterns, and forecasting traffic trends on urban roads. Due to the complex spatial and temporal dependencies involved, this problem remains challenging and is an active area of research.

The spatial dependencies in this problem arise from the fact that traffic conditions on one road segment are statistically related to conditions on other segments through the topology of the urban road network and through shared demand patterns. Traffic volume also changes dynamically over time, affected by short-term and long-term patterns, which underscores the temporal dependencies in traffic data.

These relationships motivate sophisticated models that can capture joint spatio-temporal structure. A critical limitation of many existing approaches, however, is their reliance on physical proximity or predefined road-network topology to construct the adjacency matrix. Physical connectivity need not coincide with functional co-movement in the data: distant roads can exhibit strong traffic correlations because of commuting patterns, while adjacent roads can have weak statistical dependence. Moreover, dense proximity graphs can degrade graph-convolutional models through oversmoothing, so that a graph-based forecaster can be worse than the same backbone without a graph. These observations raise a natural question: if the adjacency matrix is a modeling choice rather than a fixed physical fact, should it be estimated from the traffic data themselves?

Multiple traffic prediction methods, including neural networks, statistical approaches, and Bayesian methods, model temporal evolution but neglect spatial dependencies [1]. To describe spatial features, many studies use graph-based methods [2–5]. In [6], a Temporal Graph Convolutional Network (T-GCN) was proposed for traffic prediction. T-GCN combines a Graph Convolutional Network (GCN) with a Gated Recurrent Unit (GRU): the GCN captures spatial dependence and the GRU captures temporal dependence. The combined model achieves strong results on standard traffic benchmarks.

Existing graph-based methods typically build the adjacency matrix from road proximity or distance [6]. While such graphs capture geographic connectivity, they need not represent the statistical dependency structure present in the observed speeds. Attention-based methods [7–9] reweight relationships between nodes over time, but they usually operate on an initial graph that is still defined by heuristics or physical proximity; they change how a given structure is used more than which structure is the right one.

Our work studies *graph structure learning* (GSL) for traffic forecasting: the adjacency matrix is inferred from the traffic series rather than taken as predefined. Continuous optimization methods for directed acyclic graphs, such as NOTEARS [10] and DAGMA [11], make such estimation practical. Relative to implicit reweighting over a fixed topology, GSL produces an explicit binary (or weighted) adjacency that can be inspected as a statistical dependency graph over sensors. We interpret that graph as descriptive statistical structure under the estimator’s assumptions, not as a causal map of traffic influence.

This paper evaluates that idea under a controlled experimental design on two public benchmarks, SZ-Taxi and Los-loop. In addition to the physical road-network graph, we include a *graph-free* identity control, so that any contribution of learned structure can be measured against both a standard baseline and a model that uses no spatial graph. We consider a single contemporaneous learned graph (and its binary symmetrization, cGSL) as well as *multi-lag* learned graphs in which lag-specific structures are estimated from an explicitly lag-stacked input. Because a learned adjacency can be applied as one static operator or aligned with input timesteps in a recurrent backbone, we further examine how the *consumption* of multi-lag graphs affects forecasting accuracy. Finally, matched-sparsity controls address the methodological concern that any gain might be due only to the sparsity of the learned graph rather than to the placement of its edges.

The experimental results confirm that learning graph structure from traffic data can improve forecasting, but they also qualify the claim. Relative to the physical graph, learned and even identity adjacencies often reduce error. Relative to the graph-free control, the largest and most consistent gains on Los-loop appear when lag-specific structures are consumed in a temporally aligned manner. On SZ-Taxi, where the recovered multi-lag structure is extremely sparse, learned variants remain close to the graph-free baseline. Thus, the usefulness of a learned graph depends on the dataset and on how the learned structure is incorporated into the forecasting model; it is not a universal replacement for either the physical graph or a graph-free model.

The contributions of this paper are the following:

1. We investigate GSL for traffic forecasting with GCN and T-GCN backbones, replacing a predefined physical adjacency with an adjacency estimated from the traffic data.
2. We compare learned graphs not only to the physical baseline but also to a graph-free control, and we evaluate both a contemporaneous construction and a multi-lag construction of the learned structure.
3. We study how multi-lag graphs are consumed by the forecasting model (static union versus lag-aligned recurrent use, including a gated mixing variant), which separates the estimated structure from the mechanism that uses it.
4. We report five-seed experiments on SZ-Taxi and Los-loop, together with matched-sparsity controls, showing that the benefit of learned structure is dataset-dependent and cannot be attributed to sparsity alone on the dataset where gains are clearest.

In the following sections, we review background on traffic prediction and GCN/T-GCN models (Section 2). We then present the GSL framework used to estimate the adjacency matrix and its integration with GCN and T-GCN (Section 3). Experimental settings are described in Section 4, results in Section 5, and discussion in Section 6. Limitations are stated in Section 7, and conclusions in Section 8.

## 2 Background and Related Work

The goal of traffic forecasting is to predict future traffic conditions from historical data. Traffic conditions can include metrics such as speed, flow, or density; in our experiments we focus on traffic speed without loss of generality.

We model the road network as a graph  $G = (V, E)$ , where  $V = \{v_1, \dots, v_N\}$  represents  $N$  road segments (sensors) as nodes,  $E$  represents edges between connected roads, and  $\mathbf{A} \in \{0, 1\}^{N \times N}$  is the adjacency matrix encoding those connections (or, in learned variants, estimated statistical adjacency). The traffic data are represented by a feature matrix  $X \in \mathbb{R}^{N \times P}$ , where  $P$  is the length of the historical series and  $X_t \in \mathbb{R}^N$  is the vector of speeds at time  $t$ . The forecasting task is to learn a mapping

$$[X_{t+1}, \dots, X_{t+T}] = f(G; [X_{t-n}, \dots, X_t]), \quad (2.1)$$

where  $n$  is the number of historical steps used as input and  $T$  is the prediction horizon.

### 2.1 Overview of Traffic Prediction Approaches

Traffic prediction methods can be broadly categorized as follows.

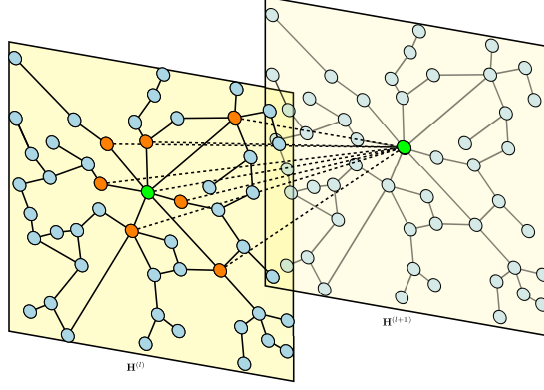
*Classical prediction methods* include ARIMA models [12, 13], support vector regression [14], seasonal ARIMA variants [15], and Kalman filtering approaches [16]. These methods are effective for basic temporal patterns on individual series but often struggle with network-scale spatial dependence and strong nonlinearity.

*Classical deep learning methods* include feedforward networks [17], spatio-temporal residual networks [18], interactive temporal recurrent convolutional networks [19], and recurrent variants such as GRU and LSTM [20, 21]. They capture rich temporal structure and some spatial coupling, but their Euclidean assumptions limit the modeling of irregular network topology.

*Graph-based and temporal graph methods* explicitly model network topology. Representative models include GCNs [22, 23], T-GCN [6], attention-based temporal GCNs (A3T-GCN) [7], dynamic spatial-temporal attention networks [8], and a range of recent spatio-temporal graph convolutional, recurrent, and transformer architectures [9, 24–28]. These methods model topology and time jointly, but they typically rely on a predefined graph that may not match the dependency structure in the data.

Comprehensive reviews are available elsewhere [26, 29, 30]. Our focus is the role of the adjacency matrix itself. A bibliometric analysis of 784 traffic-forecasting articles (2000–2025) confirms the rise of graph-based methods and the continued dominance of heuristic graph construction (Appendix ??).

Since this paper builds on GCN and T-GCN, we next summarize those two backbones concisely; fuller treatments appear in [6, 22].



**Fig. 1:** A graph convolutional architecture with layers  $l$  and  $(l+1)$  computing  $\mathbf{H}^{(l+1)}$  as in (2.2). Neighbors are defined by the adjacency matrix  $\mathbf{A}$ . In the standard setting  $\mathbf{A}$  comes from the physical road network; in this work adjacency is also estimated from traffic data.

## 2.2 Graph Convolutional Network

Graph Convolutional Networks extend convolution to graph-structured data [22, 31] and have been applied widely, including community detection [32, 33], semi-supervised learning [22, 34], and recommendation [35]. Figure 1 illustrates a graph-convolutional layer used for traffic prediction: the green node is the target sensor and orange nodes are its graph neighbors. Traditional pipelines take  $\mathbf{A}$  from the physical road network; our proposed pipeline instead estimates node adjacencies from traffic data.

The layer-wise propagation rule is [22]

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right), \quad (2.2)$$

where  $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N$  adds self-connections,  $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$ ,  $\mathbf{W}^{(l)}$  is a trainable feature map, and  $\sigma$  is a nonlinearity (typically ReLU). The normalized operator  $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$  aggregates neighborhood information while controlling scale; a two-layer GCN can be written as

$$f(\mathbf{A}, \mathbf{X}) = \mathbf{H}^{(2)} = \sigma\left(\hat{\mathbf{A}}\mathbf{H}^{(1)}\mathbf{W}^{(1)}\right). \quad (2.3)$$

In traffic applications, this aggregation encodes spatial dependence over the graph defined by  $\mathbf{A}$ ; temporal dependence is added by the recurrent component described next.

## 2.3 Temporal Graph Convolutional Network

T-GCN [6] combines GCN with a GRU for traffic forecasting. At each time  $t$ , spatial features  $\mathbf{Z}_t = f(\mathbf{A}, \mathbf{X}_t)$  are obtained from (2.3); the GRU then updates a hidden state

$\mathbf{S}_t$  using update and reset gates  $(\mathbf{u}_t, \mathbf{r}_t)$  and trainable matrices  $\mathbf{W}_u, \mathbf{W}_r, \mathbf{W}_c$ . Training minimizes the discrepancy between predicted speeds  $\hat{Y}_t$  and observed speeds  $Y_t$ , for example

$$loss = \|Y_t - \hat{Y}_t\| + \lambda L_{reg}, \quad (2.4)$$

with  $L_{reg}$  an  $\ell_2$  penalty. T-GCN therefore models spatial dependence through graph convolution and temporal dependence through gating, and is the principal spatiotemporal backbone in our experiments. The non-recurrent GCN backbone applies graph convolution over the input window and is used as a spatial counterpart.

## 2.4 Graph Structure Learning

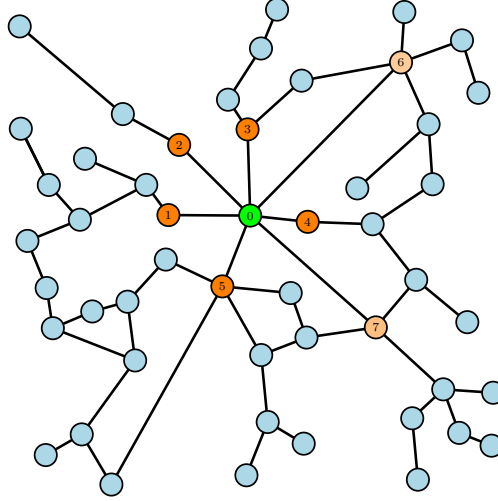
Graph structure learning estimates  $\mathbf{A}$  (or a weighted coefficient matrix  $W$ ) from observed data rather than from a predefined map [36]. In traffic, the motivation is that proximity is an imperfect proxy for functional co-movement, and that sparse data-driven graphs may avoid the oversmoothing induced by dense physical adjacencies.

Score-based continuous DAG learners replace combinatorial search over graphs with smooth optimization on real matrices. NOTEARS [10] introduces a differentiable acyclicity constraint; DAGMA [11] strengthens the approach with a log-determinant characterization suitable for efficient optimization. We use DAGMA with a linear structural-equation score and an  $\ell_1$  sparsity penalty. After fitting, a binary adjacency is obtained by thresholding absolute coefficient magnitudes. Acyclicity here is an optimization device; we do not interpret the learned graph as a causal traffic model. The precise contemporaneous and multi-lag constructions, including thresholds and edge budgets, are specified in Section 3.

Compared with attention mechanisms that reweight neighbors on a fixed topology, GSL changes the edge set itself. Compared with purely temporal models, it makes the spatial operator an object of estimation. In this paper we study that estimated operator both as a single contemporaneous graph and as lag-specific blocks, and we evaluate it against physical and graph-free references under the same forecasting protocol.

## 2.5 Position of this Work

This work remains a study of GSL for traffic prediction. Relative to the standard GCN/T-GCN pipeline with a physical adjacency, we ask whether a learned adjacency is preferable, and under what conditions. Three extensions relative to a minimal GSL baseline are essential for answering that question honestly: (i) a graph-free control, without which learned graphs cannot be separated from the effect of removing a harmful default; (ii) an explicit multi-lag construction, so that temporal organization of the learned structure is defined by the estimator’s input rather than asserted post hoc; and (iii) an analysis of how multi-lag graphs are consumed by recurrent versus non-recurrent backbones. These elements do not change the paper’s identity as a graph-structure-learning study; they specify the conditions under which learned structure helps.



**Fig. 2:** Conceptual two-dimensional traffic network. Association between a target node (green) and other nodes depends not only on spatial proximity (lighter color indicates larger distance) but also on topology and travel time. The learned adjacency aims to reflect such data-driven associations rather than geographic distance alone.

### 3 The Proposed Method: Estimating the Adjacency Matrix with Graph Structure Learning

In both GCN [22] and T-GCN [6], the adjacency matrix  $\mathbf{A}$  that describes spatial relationships among sensors is typically predefined from urban planning or road-network information. Here we estimate  $\mathbf{A}$  from traffic data, which is known as *graph structure learning* (GSL) [36]. The goal is to obtain a graph representation that reflects dependency structure present in the observed series rather than structure imposed solely by physical proximity.

#### 3.1 Motivation

In a typical urban traffic network, spatial proximity of roads need not coincide with statistical dependence among their speeds. Distant segments can be strongly correlated through shared demand, while adjacent segments can be weakly related. Propagation delay is also relevant: association between two sensors may appear only at particular temporal lags. Estimating the adjacency directly from the data allows the forecasting model to aggregate from neighbors defined by observed co-movement rather than only by the map.

Figure 2 illustrates a stylized network related to Figure 1. Three points motivate data-driven adjacency: (i) nearby nodes may share more immediate co-movement than distant ones, but this need not hold for every pair; (ii) associations between distant nodes may occur at longer lags; (iii) some physically adjacent links may carry little predictive information for the target sensor. By learning  $\mathbf{A}$  from traffic observations,

the model can place edges according to estimated dependency rather than geographic distance alone.

Throughout this section,  $\mathbf{A} \in \{0, 1\}^{N \times N}$  denotes the binary adjacency supplied to the forecasting backbone (Eqs. (2.2)–(2.3)), and  $W \in \mathbb{R}^{d \times d}$  denotes the continuous coefficient matrix estimated by the graph-learning procedure. The two symbols are not interchangeable:  $W$  is the estimator’s internal representation;  $\mathbf{A}$  is the thresholded graph used by GCN or T-GCN.

### 3.2 Integration of Learned Structure with Temporal Modeling

The overall pipeline separates three stages:

- **Graph estimation (GSL).** A static adjacency is estimated from training traffic data (contemporaneous or multi-lag constructions below).
- **Normalization.** Every adjacency used in this paper — physical, identity (graph-free control), and learned — is passed through the same GCN-style operator

$$\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}}^\top \tilde{\mathbf{D}}^{-1/2}, \quad \tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}, \quad \tilde{D}_{ii} = \sum_j \tilde{A}_{ij}. \quad (3.1)$$

For the identity adjacency this reduces to  $\hat{\mathbf{A}} = \mathbf{I}$ , so the graph-free control differs only in the absence of off-diagonal structure.

- **Forecasting backbone.** GCN or T-GCN consumes the normalized operator. The learned graph is *static* after estimation: it is fitted once on the training split. Temporal variation in the forecast is modeled by the recurrent backbone (and, in one variant, by a gate that selects among fixed lag-specific graphs); the edge set itself does not change over time.

Two reference adjacencies accompany the learned graphs in all experiments: the *physical* road-network graph (standard T-GCN/GCN baseline) and a *graph-free* identity control (T-GCN-NoSpatial / GCN-NoSpatial). The latter is required to measure whether learned structure adds value beyond removing a possibly harmful dense proximity graph.

### 3.3 Continuous Optimization for Graph Structure Learning

Score-based GSL seeks a weighted adjacency that fits the data under a structural equation model (SEM). Following the continuous formulation of Zheng et al. [10], let  $\mathbf{X} \in \mathbb{R}^{n \times d}$  be a data matrix with  $n$  observations of  $d$  variables. In traffic, rows of  $\mathbf{X}$  are time indices and columns are sensors. Instead of searching the discrete DAG space, one optimizes over  $W \in \mathbb{R}^{d \times d}$  with a smooth acyclicity constraint [10]:

$$\begin{aligned} \min_W \quad & F(W) \\ \text{subject to} \quad & h(W) = 0, \end{aligned} \quad (3.2)$$



where  $F$  measures data fit and  $h$  vanishes exactly on acyclic graphs. NOTEARS uses

$$h(W) = \text{tr}(e^{W \circ W}) - d, \quad (3.3)$$

with gradient  $\nabla h(W) = (e^{W \circ W})^\top \circ 2W$ , and solves the program with an augmented Lagrangian

$$L^\rho(W, \alpha) = F(W) + \frac{\rho}{2} |h(W)|^2 + \alpha h(W). \quad (3.4)$$

After optimization, a binary support is obtained by absolute-magnitude thresholding,  $\widehat{W} = W \circ \mathbf{1}(|W| > \omega)$ . Algorithmic details appear in [10].

***Estimator used in this paper (DAGMA).***

In the experiments we use **DAGMA** [11], which replaces the NOTEARS trace-exponential constraint with a log-determinant acyclicity characterization suitable for efficient constrained optimization, together with a linear SEM score ( $\ell_2$  reconstruction loss) and an  $\ell_1$  sparsity penalty  $\lambda_1 \|W\|_1$ . Acyclicity is an *optimization regularizer* of the estimator. We interpret the learned graph as a statistical dependency structure under the linear SEM assumptions, not as a causal traffic model. DAGMA is applied in two related ways: a *contemporaneous* construction (single graph among sensors; Section 3.4) and a *multi-lag* construction (lag-specific blocks; Section 3.5).

### 3.4 Contemporaneous Graph Learning (GSL and cGSL)

The contemporaneous construction is a single learned graph among the  $N$  sensors from simultaneous observations — the direct analogue of the original GSL-for-traffic idea.

***Input and fit.***

DAGMA is fitted to the training-split snapshot matrix

$$\mathbf{X} = \text{train\_norm}[0 :: \text{PH}],$$

i.e. every PH-th training row (one simultaneous vector of  $N$  sensor speeds per row; no lag blocks). For Los-loop the row counts are 1612, 806, 538, and 403 for  $\text{PH} = 1, \dots, 4$ ; for SZ-Taxi, 2380, 1190, 794, and 595. The sparsity coefficient is  $\lambda_1 = 0.02$  (Los-loop) and  $\lambda_1 = 0.01$  (SZ-Taxi). DAGMA’s internal magnitude threshold is 0.3. The binary adjacency is

$$\mathbf{A}_{\text{GSL}} = \mathbf{1}(|W| > 0)$$

after that internal threshold (absolute-magnitude rule; diagonal removed), using training data only. Edge counts are 28 (Los-loop) and 8 (SZ-Taxi) at every horizon.

***cGSL (symmetrized GSL).***

The cyclic variant cGSL is not a second fit. It is the binary symmetrization of the same stored GSL artifact:

$$\mathbf{A}_{\text{cGSL}} = \mathbf{1}(\mathbf{A}_{\text{GSL}} + \mathbf{A}_{\text{GSL}}^\top > 0), \quad (3.5)$$

with the diagonal removed (56 directed entries on Los-loop, 16 on SZ-Taxi). The same artifacts serve both GCN and T-GCN counterparts, so any difference between backbones is not due to a different learned edge set.

***Interpretation.***

The contemporaneous graph is estimated from simultaneous snapshots. An edge is a statistical association among present observations; it does not encode “sensor  $j$  at time  $t$  predicts sensor  $i$  at time  $t + 1$ .” Temporal organization of learned structure is introduced explicitly in the multi-lag construction next, rather than inferred from this DAG.

### 3.5 Multi-Lag Graph Learning

A single contemporaneous graph cannot distinguish associations at different temporal scales. We therefore estimate lag-specific structure from an explicitly lag-stacked input, still within the same DAGMA/GSL framework.

With lag depth  $L = 3$ ,

$$Z = [x(t-3), x(t-2), x(t-1), x(t)],$$

stacked over training windows (1609 windows on Los-loop; 2377 on SZ-Taxi). The variable dimension is  $(L + 1)N$ : 828 (Los-loop) or 624 (SZ-Taxi). DAGMA is fitted *once* on the training split; the artifact is shared across  $\text{PH} = 1, \dots, 4$  (unlike the contemporaneous input, which is PH-subsampled).

Here  $\lambda_1 = 0.01$  on both datasets, the internal threshold is 0.0 (raw weights retained), and the forecasting consumer thresholds at  $|W| > 0.1$  (absolute magnitude; diagonal removed). In the DAGMA convention,  $W_{ij}$  quantifies estimated dependence of variable  $j$  on variable  $i$ ; the block mapping lag- $k$  variables to the current-time block yields lag adjacencies  $\mathbf{A}_1, \mathbf{A}_2, \mathbf{A}_3$ . The contemporaneous block inside  $W$  is not consumed by the forecasting methods.

After consumer thresholding, per-lag off-diagonal counts are 12/3/15 (sum 30 slots; union 28 distinct edges) on Los-loop and 0/0/2 (union 2) on SZ-Taxi. The lag-structured reading is consistent with the construction; we do not independently validate it as a physical propagation model, and acyclicity remains a fitting constraint only. The graphs are static after fitting.

### 3.6 Using Learned Graphs in GCN and T-GCN

***Single-graph use (GSL / cGSL / physical / identity).***

One normalized adjacency is applied for the whole input window in GCN, and at every recurrent step in T-GCN. This is the standard integration of a learned static structure with temporal modeling: GSL provides persistent spatial dependency estimates; the backbone models temporal evolution on that fixed operator.

### *Multi-lag use in T-GCN.*

When three lag graphs are available, T-GCN can apply them in different ways. We evaluate:

- **Fixed lag assignment (T-GCN-MultiGSL).** At input index  $t$  ( $t = 0$  most recent), the graph index is

$$\text{graph\_idx} = (T - 1 - t) \bmod 3, \quad (3.6)$$

so recent steps use lag-1 and older steps cycle through lag-2 and lag-3. No extra parameters.

- **Global mixture (T-GCN-MultiGSL-Weighted, supplementary).** Three learned scalars (softmax) mix the normalized lag operators for every step (+3 parameters).
- **Per-node gate (T-GCN-MultiGSL-Mix).** At each step  $t$  and node, a small MLP on  $[x_t; h_{t-1}]$  produces a softmax over the  $K = 3$  graphs; the mixed operator is applied before the GRU update (+4,419 parameters). Mix changes how the *static* lag graphs are used per node and timestep; it does not relearn the edge set.

### *Multi-lag use in GCN (GCN-MultiGSL).*

The non-recurrent GCN applies a single operator to the window, so lag graphs are merged into a static union

$$\mathbf{A}_{\cup} = \bigvee_{k=1}^3 \mathbf{A}_k$$

(element-wise OR), then normalized as in (3.1). Comparing GCN-MultiGSL with T-GCN-MultiGSL asks whether the same multi-lag artifacts are useful when consumed as one static graph versus per timestep. Because the backbones also differ, this comparison is evidence about consumption under different architectures, not a fully isolated causal test of consumption alone.

### *Capacity.*

Within a backbone family, graph sources add no trainable parameters. Verified counts (excluding the shared output layer) are 768 for GCN and 12,672 for single-graph T-GCN; Mix adds 4,419 relative to T-GCN-MultiGSL. Capacity-matched checks are reported with the experiments (Section 4).

## 3.7 Summary of Graph Sources

Table 1 summarizes the adjacencies compared in this paper. The experimental study asks whether replacing the physical graph with a learned one improves forecasting, whether a graph-free control changes that assessment, and — for multi-lag structure — how the learned graphs should be used inside the backbone.

**Table 1:** Graph sources used in the experiments. All are normalized by Eq. (3.1). “Static” means the adjacency is fixed after estimation; temporal modeling is in the backbone (and in Mix, in the *use* of fixed lag graphs).

| Name            | Origin       | Construction                  | Use                     |
|-----------------|--------------|-------------------------------|-------------------------|
| Physical        | road network | dataset adjacency             | static                  |
| NoSpatial       | none         | identity $\mathbf{I}$         | static                  |
| GSL             | DAGMA        | contemporaneous               | static                  |
| cGSL            | DAGMA        | (3.5) of GSL                  | static                  |
| MultiGSL family | DAGMA        | lag blocks $\mathbf{A}_{1:3}$ | per-step / gate / union |

## 4 Experimental Setup

This section describes the complete experimental protocol. All twelve forecasting configurations, the auxiliary controls, and the temporal-resolution variant share the same data handling, training procedure, and evaluation.

### 4.1 Datasets

Two widely used public traffic benchmarks are used [6]:

- **Los-loop.** Speed measurements from 207 highway loop detectors on Los Angeles County highways, collected in March 2012 and aggregated at 5-minute intervals; the series comprises 2016 timesteps (seven days).
- **SZ-Taxi.** Speed measurements from 156 urban road sensors in the Luohu District of Shenzhen, China, collected in January 2015 and aggregated at 15-minute intervals; the series comprises 2976 timesteps.

Each dataset provides a feature matrix (sensor speeds over time) and a road-network adjacency, which is used for the physical baseline (Sect. ??).

### 4.2 Data Split and Normalization

The series are split chronologically into 80% training and 20% test segments: 1612 training and 404 test timesteps for Los-loop, and 2380 training and 596 test timesteps for SZ-Taxi. Input windows and targets are generated separately within each segment, so no forecasting window crosses the train/test boundary.

Features are normalized by dividing by the *training-split* maximum only: 70.0 (km/h) for Los-loop and 86.4292 for SZ-Taxi. The test split is not used for normalization or graph learning.

### 4.3 Forecasting Protocol

The input window length is  $T = 12$  and the prediction horizons are  $\text{PH} \in \{1, 2, 3, 4\}$ , expressed in steps of the dataset’s sampling interval and produced as the multi-step target defined in Sect. ?. In wall-clock terms the horizons correspond to:

- **Los-loop** (5-minute steps): PH1 = 5 min, PH2 = 10 min, PH3 = 15 min, PH4 = 20 min ahead.
- **SZ-Taxi** (15-minute steps): PH1 = 15 min, PH2 = 30 min, PH3 = 45 min, PH4 = 60 min ahead.

Because the two datasets use different sampling intervals, horizon indices are only comparable within, not across, datasets.

#### 4.4 Methods

The study evaluates twelve configurations: seven in the T-GCN family and five in the GCN family, spanning the graph sources and consumption mechanisms defined in Sect. 3.

##### *T-GCN family.*

T-GCN (physical graph), T-GCN-NoSpatial (graph-free control), T-GCN-GSL, T-GCN-cGSL (learned contemporaneous graphs, static consumption), and three consumption mechanisms over the learned multi-lag graphs: T-GCN-MultiGSL (fixed cyclic lag assignment), T-GCN-MultiGSL-Weighted (global lag mixture), and T-GCN-MultiGSL-Mix (per-node, per-timestep gate).

##### *GCN family.*

GCN (physical graph), GCN-NoSpatial (graph-free control), GCN-GSL, GCN-cGSL (learned contemporaneous graphs), and GCN-MultiGSL (static union of the multi-lag graphs).

All configurations use architecturally identical backbones within each family and differ only in the adjacency supplied and, for the multi-lag T-GCN variants, in how the lag graphs are consumed (Sect. 3.6).

#### 4.5 Training Protocol

All forecasting models are trained with Adam (learning rate  $10^{-3}$ , weight decay  $10^{-4}$ ), batch size 128, for 50 epochs, with hidden dimension 64 and a shared linear output layer mapping the hidden state to the PH-dimensional target.

The training loss differs by backbone family, as in the reference implementations of the two architectures. The T-GCN family is trained with an  $\ell_2$  loss combined with an  $\ell_2$  penalty on the model weights,

$$\mathcal{L}_{\text{T-GCN}} = \frac{1}{2} \sum_i (\hat{y}_i - y_i)^2 + \lambda_{\text{reg}} \cdot \frac{1}{2} \sum_{\theta \in \Theta} \|\theta\|_2^2, \quad \lambda_{\text{reg}} = 1.5 \times 10^{-3},$$

where the sums run over prediction elements and over all trainable weights; the GCN family is trained with the plain mean squared error. Both families share the same output layer and optimizer settings.

Each configuration is trained under five seeds,  $\{42, 43, 44, 45, 46\}$ , and all reported statistics aggregate over these five training runs. The learned graphs are fitted once

and are deterministic; they are shared across the five forecasting-model training seeds, so seed variability reflects the training and evaluation of the forecasting models only.

Graph learning uses DAGMA-linear [11] with the  $\ell_2$  loss type, 30,000 warm-up iterations and 60,000 maximum iterations (library defaults), and internal seed 42; the estimator is deterministic (zero initialization, no stochastic component) under this configuration. All graph-learning hyperparameters and threshold regimes are as specified in Sects. 3.4 and 3.5.

## 4.6 Evaluation Metrics

The primary metric is the root mean squared error (RMSE); the mean absolute error (MAE) is reported as a secondary metric. Both are computed on the de-normalized original speed scale. Test evaluation is performed on the complete test set in a single full-batch pass.

## 4.7 Statistical Reporting

Main-text numbers are five-seed means with sample standard deviations (ddof = 1). We report paired per-seed win counts and paired  $t$ -tests for reference. With  $n = 5$ , the exact Wilcoxon signed-rank test cannot fall below  $p = 0.0625$ ; we therefore make no definitive significance claims.

## 4.8 Matched-Sparsity and Capacity Controls

Two supplementary controls probe alternative explanations of the multi-lag results. Both use the same training protocol as the main experiments, and both are scope-labeled: they are attached to a single experimental cell rather than to the complete configuration matrix, and their conclusions do not extend beyond that cell.

### *Matched sparsity (Los-loop, PH = 1).*

On the Los-loop PH1 cell, the multi-lag construction consumes 30 lagged edge slots. This control compares, at the same 30-edge budget: RandTop30, a graph of 30 uniformly random directed edges, redrawn per seed; CorrTop30, the 30 largest-magnitude pairwise Pearson correlations of the training data; and the DAGMA multi-lag graphs under the fixed and the gated consumption mechanisms. Five seeds are used per graph. This control cell lies outside the complete 12-method dataset  $\times$  horizon matrix and is reported separately. No sparsified variant of the physical graph and no sensitivity sweep over sparsity coefficients or thresholds was performed.

### *Capacity control.*

The gated mechanism adds 4,419 parameters relative to T-GCN-MultiGSL (Sect. 3.6). To check that its results are not explained by capacity alone, a hidden-dimension-74 T-GCN-NoSpatial variant (16,872 parameters, matching the 17,091-parameter gated model more closely than the 12,672-parameter standard T-GCN) is compared against the hidden-64 NoSpatial and the gated model on the same cell. This check is single-seed and is labeled as such.

## 4.9 Temporal-Resolution Variant

To probe the effect of temporal resolution, the Los-loop series is aggregated by averaging consecutive groups of three 5-minute observations, yielding a 15-minute series of length  $T = 672$  with an 80/20 split. Prediction horizons  $\text{PH} \in \{1, 2, 3, 4\}$  correspond to 15, 30, 45, and 60 minutes ahead. This is a *temporal-resolution variant* of Los-loop, not a third dataset, and its PH indices must not be interpreted as equivalent to PH indices in the original 5-minute Los-loop series. A separate PH-independent multi-lag DAGMA fit is performed on this training split ( $\lambda_1 = 0.01$ ; consumer threshold  $|W| > 0.1$ ), and three configurations are trained per horizon with five seeds under the standard protocol: T-GCN-NoSpatial, T-GCN-MultiGSL, and T-GCN-MultiGSL-Mix.

## 4.10 Implementation and Reproducibility

The implementation, data, and configuration files are available at <https://github.com/mamintoosi/TGCN-GSL-PyTorch>. Graph learning uses training data only and has no access to the test split, for both the contemporaneous and the multi-lag constructions and for all control graphs.

# 5 Results

This section reports the forecasting results under the protocol of Sect. 4. Unless stated otherwise, every value is the mean RMSE over the five training seeds  $\{42, \dots, 46\}$  with the sample standard deviation in parentheses; graphs are fixed across seeds. Improvements are relative RMSE reductions with respect to the named reference. Statistical qualifications follow Sect. 4.7: with  $n = 5$  we do not make definitive significance claims. Interpretation is deferred to the Discussion.

## 5.1 The Physical Graph Is Not a Sound Default

Table 2 reports the T-GCN family on both datasets. The physical road-network graph (T-GCN) is the worst configuration in its family in every dataset–horizon cell. On Los-loop at PH1, T-GCN attains  $7.88 \pm 0.28$  RMSE, whereas the graph-free control T-GCN-NoSpatial attains  $5.25 \pm 0.19$  — a relative RMSE reduction of 33.3% when the physical graph is removed. The same direction holds at PH2–PH4 on Los-loop (29.2%, 27.0%, and 24.1% relative to T-GCN) and at every horizon on SZ-Taxi (24.4%, 25.1%, 25.2%, 25.1%). Across all eight dataset  $\times$  horizon cells, NoSpatial beats Physical in 5/5 seeds.

The GCN family (Table 3) shows the same reversal: GCN (physical) attains  $8.14 \pm 0.11$  on Los-loop PH1 versus  $4.88 \pm 0.31$  for GCN-NoSpatial, and remains worse than GCN-NoSpatial at every horizon on both datasets.

These results establish the graph-free control, not the physical graph, as the appropriate reference for evaluating learned structure at these operating points (structural contrast in Fig. 5).

## 5.2 A Single Learned Graph Does Not Close the Gap

The contemporaneous learned graphs GSL and cGSL (Sects. 3.4) improve on the physical graph in most T-GCN cells, but they do not close the gap to the graph-free control.

On Los-loop, T-GCN-GSL and T-GCN-cGSL obtain  $5.86 \pm 0.21$  and  $5.82 \pm 0.23$  at PH1, versus  $5.25 \pm 0.19$  for NoSpatial: both are approximately 11% *worse* than removing the graph, while still reducing error by roughly 26% relative to the physical baseline. The same ordering — Physical > single learned graph > NoSpatial in error — holds at PH2–PH4 on Los-loop and at every horizon on SZ-Taxi. In no dataset  $\times$  horizon cell does either contemporaneous learned graph beat the graph-free control.

The GCN family is less uniform. GCN-cGSL ( $5.76 \pm 0.20$  on Los-loop PH1) beats the physical GCN but still loses to GCN-NoSpatial ( $4.88 \pm 0.31$ ). GCN-GSL is the notable exception: on Los-loop it is often comparable to, or worse than, the physical GCN ( $7.83 \pm 0.11$  at PH1) and does not consistently improve on that baseline. Symmetrization therefore helps the non-recurrent GCN backbone (cGSL < GSL in error on Los-loop PH1: 5.76 vs 7.83) while the directed and symmetrized variants are nearly indistinguishable for T-GCN ( $5.86$  vs  $5.82$ ). Mechanism-level discussion of this asymmetry is deferred to Sect. 6.

In summary, replacing the physical adjacency with a single learned contemporaneous graph recovers only part of the error reduction obtained by removing the graph entirely. Motivated by this gap, the remainder of the results examine lag-specific multi-lag graphs and, critically, how those graphs are consumed.

## 5.3 Sparsity and Capacity Controls

Before interpreting any multi-lag gain, we report the matched-sparsity and capacity controls of Sect. 4.8. These controls exist **only for Los-loop at PH1** and do not establish conclusions for other cells.

At a matched budget of 30 directed edges, the random control RandTop30 obtains  $6.10 \pm 0.12$  and the top-correlation control CorrTop30 obtains  $5.39 \pm 0.10$ . Both are worse than the graph-free control on the same cell ( $5.25 \pm 0.19$ ). By contrast, the DAGMA multi-lag graphs at the same 30-edge budget obtain  $4.84 \pm 0.11$  under fixed per-timestep consumption (T-GCN-MultiGSL) and  $4.49 \pm 0.14$  under gated consumption (T-GCN-MultiGSL-Mix). Table 4 summarizes the comparison.

These matched-budget results show that sparsity alone does not explain the Los-loop PH1 multi-lag gain: two non-learned 30-edge graphs are worse than no graph, whereas DAGMA-placed edges with per-lag consumption are better. The conclusion is scoped to this single experimental cell.

The capacity control (single seed, seed 42, labeled as such) compares T-GCN-NoSpatial at hidden dimension 64 (12,672 parameters, RMSE 5.14) with a hidden-dimension-74 variant (16,872 parameters, RMSE 5.14) that more closely matches the parameter count of T-GCN-MultiGSL-Mix (17,091). Increasing capacity on the graph-free model changes RMSE by less than 0.02, whereas Mix reaches 4.49 at five seeds on the same cell. The Mix gain is therefore not explained by its additional 4,419 gate parameters alone.



## 5.4 Multi-Lag Graphs: Consumption Matters

We next compare configurations that use the *same* multi-lag DAGMA artifacts (Sect. 3.5) under different consumption mechanisms (Sect. 3.6).

On Los-loop at PH1, T-GCN-MultiGSL (fixed cyclic lag assignment; 0 extra parameters) obtains  $4.84 \pm 0.11$ , improving on NoSpatial by 7.8%. T-GCN-MultiGSL-Weighted, which learns only three global scalars, obtains  $4.83 \pm 0.11$  — effectively identical to the fixed assignment. The global lag mixture does not add value beyond the fixed per-timestep rule.

The striking comparison is with GCN-MultiGSL, which consumes the *identical* three lag graphs as one static union  $\mathbf{A}_\cup = \bigvee_k \mathbf{A}_k$  (28 of the 30 lagged edge slots appear in the union). GCN-MultiGSL obtains  $9.78 \pm 0.14$  on Los-loop PH1: worse than the physical GCN ( $8.14 \pm 0.11$ ) and more than twice the error of T-GCN-MultiGSL on the same graphs ( $4.84 \pm 0.11$ ). The direction persists at PH2–PH4 (GCN-MultiGSL 10.05–10.27 versus T-GCN-MultiGSL 5.45–6.26).

Because the union holds nearly the same edges as the three lag graphs, the edge set alone cannot account for the difference between the two outcomes (Fig. 3). At the same time, GCN and T-GCN also differ in backbone architecture, so this comparison provides evidence that *how the multi-lag structure is consumed* is a major determinant of whether it is useful; it is not a fully controlled isolation of consumption as the sole differing factor. On SZ-Taxi, where the multi-lag graphs retain only 2 edges, the corresponding contrast is small in absolute RMSE.

## 5.5 Gated Mixing on Los-Loop: The Positive Result

T-GCN-MultiGSL-Mix, which gates the three static lag graphs per node and per input timestep, is the strongest configuration on Los-loop at every horizon (Table 2). Relative to the graph-free control, Mix reduces RMSE by 14.5%, 11.9%, 9.2%, and 10.9% at PH1–PH4. Relative to the physical T-GCN baseline, the corresponding reductions are 43.0%, 37.6%, 33.7%, and 32.3%.

At PH1, Mix attains  $4.49 \pm 0.14$  versus  $5.25 \pm 0.19$  for NoSpatial. The paired per-seed difference is positive in 5/5 seeds at every horizon (mean paired difference 0.76, 0.68, 0.56, and 0.72 RMSE at PH1–PH4; paired- $t$   $p \leq 0.0015$  at PH1–PH2 and  $p \leq 0.00002$  at PH4, reported for reference under the  $n = 5$  caveat of Sect. 4.7). Mix also improves on the fixed MultiGSL assignment in 5/5 seeds at every horizon, by 0.35–0.40 RMSE on Los-loop (approximately 6–7% relative). Weighted remains within 0.01 RMSE of Fixed at every Los-loop horizon and is treated as a null ablation.

Figure 4 shows the per-seed distributions for NoSpatial, MultiGSL, Weighted, and Mix on Los-loop PH1–PH4; the means match Table 2.

This is the study’s main positive result: on Los-loop, lag-specific learned graphs improve on the graph-free baseline only when they are consumed in a temporally aligned manner, and the per-node gate adds a further consistent gain over the fixed assignment.

## 5.6 Boundaries: Temporal Resolution and Dataset Dependence

### *SZ-Taxi.*

On SZ-Taxi, every learned variant remains within seed variability of the graph-free control. At PH1, T-GCN-MultiGSL-Mix obtains  $4.119 \pm 0.020$  versus  $4.120 \pm 0.004$  for NoSpatial (difference  $\leq 0.015$  RMSE at every horizon; Mix wins 4/5, 4/5, 5/5, and 4/5 paired seeds at PH1–PH4). Relative RMSE reductions versus NoSpatial are at most 0.34%. The learned multi-lag graphs on SZ-Taxi retain only 2 lagged edges (union 2), versus 30 lagged slots (union 28) on Los-loop. The Los-loop positive result therefore does not transfer to SZ-Taxi under the present construction.

### *Temporal-resolution variant of Los-loop.*

Resampling Los-loop to 15-minute intervals (Sect. 4.9) preserves and strengthens the relative improvement of Mix over the graph-free baseline at matched wall-clock horizons of 15–60 minutes. At PH1 of the 15-minute series (15 minutes ahead), NoSpatial obtains  $8.600 \pm 0.249$ , MultiGSL  $7.246 \pm 0.298$ , and Mix  $6.240 \pm 0.187$  (a 27.4% relative RMSE reduction versus NoSpatial; Mix wins 5/5 seeds). The relative reduction persists at PH2–PH4 of the variant (21.4%, 19.8%, 16.1%). Absolute RMSE magnitudes are not comparable across sampling intervals, and PH indices of the variant are not equivalent to PH indices of the 5-minute series. Full values are collected in Table 5.

### *Summary of the boundary.*

Learned multi-lag structure with lag-aligned consumption helps where the learned graphs are informative (Los-loop at both sampling intervals). Where the learned graphs are nearly empty (SZ-Taxi), all learned variants collapse to the graph-free baseline.

## 6 Discussion

The results separate two design questions that are often conflated in graph-based traffic forecasting: *which graph is supplied to the backbone*, and *how that graph is used over the input window*. The discussion below reads the empirical pattern under that separation. We keep interpretation at the level of statistical dependency and architecture; we do not claim causal traffic mechanisms.

### 6.1 Why Dense Graphs Fail and What Learned Structure Adds

The physical road-network graph is a harmful default for these backbones at the operating points we study. On Los-loop the physical adjacency is dense (order  $10^3$  entries including self-loops for 207 sensors), yet the graph-free identity control reduces RMSE by roughly a quarter to a third relative to that graph at every horizon and on both datasets (Sects. 5.1 and 2). A simple reading is *oversmoothing* or, more generally, harmful aggregation: when almost every sensor is treated as a neighbor of every other, the graph convolution mixes signals that are not jointly informative for forecasting, and the recurrent or non-recurrent backbone cannot recover node-specific dynamics from the smeared representation. Identity adjacency freezes that mixing without removing

**Table 2:** T-GCN family: test RMSE (mean, sample standard deviation in parentheses) over five seeds. Physical: road-network graph. NoSpatial: identity adjacency. GSL/cGSL: learned contemporaneous graphs. MultiGSL family: learned multi-lag graphs under fixed, global-weighted, and gated consumption (Sect. 3).

| Method                  | PH1                | PH2                | PH3                | PH4                |
|-------------------------|--------------------|--------------------|--------------------|--------------------|
| <i>Los-loop (5 min)</i> |                    |                    |                    |                    |
| T-GCN (Physical)        | 7.88 (0.28)        | 8.13 (0.09)        | 8.37 (0.17)        | 8.66 (0.16)        |
| T-GCN-NoSpatial         | 5.25 (0.19)        | 5.76 (0.08)        | 6.11 (0.07)        | 6.58 (0.12)        |
| T-GCN-GSL               | 5.86 (0.21)        | 6.30 (0.06)        | 6.66 (0.12)        | 7.04 (0.09)        |
| T-GCN-cGSL              | 5.82 (0.23)        | 6.35 (0.12)        | 6.67 (0.11)        | 7.09 (0.12)        |
| T-GCN-MultiGSL          | 4.84 (0.11)        | 5.45 (0.14)        | 5.94 (0.03)        | 6.26 (0.03)        |
| T-GCN-MultiGSL-Weighted | 4.83 (0.11)        | 5.44 (0.13)        | 5.93 (0.03)        | 6.25 (0.03)        |
| T-GCN-MultiGSL-Mix      | <b>4.49 (0.14)</b> | <b>5.08 (0.15)</b> | <b>5.55 (0.16)</b> | <b>5.86 (0.07)</b> |
| <i>SZ-Taxi (15 min)</i> |                    |                    |                    |                    |
| T-GCN (Physical)        | 5.45 (0.11)        | 5.55 (0.08)        | 5.60 (0.05)        | 5.63 (0.06)        |
| T-GCN-NoSpatial         | 4.12 (0.00)        | 4.16 (0.01)        | 4.19 (0.01)        | 4.23 (0.01)        |
| T-GCN-GSL               | 4.28 (0.05)        | 4.31 (0.03)        | 4.33 (0.01)        | 4.37 (0.02)        |
| T-GCN-cGSL              | 4.30 (0.03)        | 4.34 (0.02)        | 4.38 (0.03)        | 4.41 (0.01)        |
| T-GCN-MultiGSL          | 4.13 (0.02)        | 4.16 (0.00)        | 4.20 (0.01)        | 4.22 (0.00)        |
| T-GCN-MultiGSL-Weighted | 4.13 (0.02)        | 4.16 (0.00)        | 4.20 (0.01)        | 4.22 (0.00)        |
| T-GCN-MultiGSL-Mix      | 4.12 (0.02)        | 4.15 (0.01)        | 4.18 (0.00)        | 4.22 (0.01)        |

the temporal path, which is consistent with the large, seed-stable gap between Physical and NoSpatial.

Learned single graphs (GSL, cGSL) occupy an intermediate position. They shrink the physical graph to a much smaller support (e.g. 28 contemporaneous edges on Los-loop) and usually improve on Physical, but they still lose to NoSpatial in every dataset×horizon cell (Sect. 5.2). Sparsity alone is not sufficient: at a matched 30-edge budget, random and top-correlation graphs are worse than no graph, while DAGMA-placed multi-lag edges with per-lag consumption are better (Sect. 5.3, Los-loop PH1 only). What the single learned graphs appear to add, relative to Physical, is a less harmful neighborhood; what they fail to add, relative to NoSpatial, is enough useful structure to pay for any residual mis-specification. The multi-lag experiments ask whether that residual can be recovered when dependency structure is allowed to be lag-specific and is consumed in a lag-aligned way.

This reading is consistent with the proximity-versus-dependency motivation: physical connectivity is an imperfect proxy for functional co-movement, and a dense proxy can be worse than no proxy. It does not imply that graph-based forecasting is generally harmful; it implies that the default reference graph should be tested against a graph-free control before learned structure is credited with gains.

**Table 3:** GCN family: test RMSE (mean, sample standard deviation in parentheses) over five seeds. GCN-MultiGSL consumes the union of the three multi-lag graphs as one static adjacency (Sect. 3.6).

| Method                  | PH1         | PH2          | PH3          | PH4          |
|-------------------------|-------------|--------------|--------------|--------------|
| <i>Los-loop (5 min)</i> |             |              |              |              |
| GCN (Physical)          | 8.14 (0.11) | 8.54 (0.28)  | 8.76 (0.26)  | 8.76 (0.29)  |
| GCN-NoSpatial           | 4.88 (0.31) | 5.60 (0.39)  | 6.02 (0.28)  | 6.26 (0.26)  |
| GCN-GSL                 | 7.83 (0.11) | 8.48 (0.21)  | 8.45 (0.19)  | 8.93 (0.12)  |
| GCN-cGSL                | 5.76 (0.20) | 6.33 (0.32)  | 6.68 (0.25)  | 6.88 (0.21)  |
| GCN-MultiGSL            | 9.78 (0.14) | 10.05 (0.16) | 10.24 (0.13) | 10.27 (0.11) |
| <i>SZ-Taxi (15 min)</i> |             |              |              |              |
| GCN (Physical)          | 5.96 (0.01) | 5.98 (0.00)  | 5.99 (0.00)  | 6.00 (0.00)  |
| GCN-NoSpatial           | 4.11 (0.00) | 4.15 (0.00)  | 4.19 (0.00)  | 4.22 (0.00)  |
| GCN-GSL                 | 4.88 (0.01) | 4.91 (0.00)  | 4.93 (0.01)  | 4.96 (0.00)  |
| GCN-cGSL                | 4.64 (0.00) | 4.67 (0.00)  | 4.70 (0.00)  | 4.73 (0.00)  |
| GCN-MultiGSL            | 4.82 (0.01) | 4.85 (0.00)  | 4.88 (0.01)  | 4.90 (0.00)  |

**Table 4:** Matched-sparsity controls on Los-loop at PH1 only (Sect. 4.8). All graphs use a 30-edge budget under the canonical T-GCN protocol; five seeds except the capacity row, which is single-seed (seed 42) and labeled as such.

| Configuration              | RMSE        | Edges / extra params | vs NoSpatial  |
|----------------------------|-------------|----------------------|---------------|
| RandTop30                  | 6.10 (0.12) | 30 random            | +16.1%        |
| CorrTop30                  | 5.39 (0.10) | 30 top- Pearson      | +2.6%         |
| T-GCN-NoSpatial            | 5.25 (0.19) | identity             | —             |
| T-GCN-MultiGSL             | 4.84 (0.11) | 30 DAGMA lag slots   | −7.8%         |
| T-GCN-MultiGSL-Mix         | 4.49 (0.14) | same graphs, gated   | −14.5%        |
| NoSpatial $h=64$ (seed 42) | 5.14        | 12,672 params        | —             |
| NoSpatial $h=74$ (seed 42) | 5.14        | 16,872 params        | $\approx 0\%$ |
| Mix (seed 42)              | 4.46        | 17,091 params        | −13.3%        |

## 6.2 Structure Versus Consumption

The central architectural observation is the dissociation in Sect. 5.4. The three lag-specific DAGMA graphs are fitted once and shared. Consumed as a static union by GCN-MultiGSL, they yield  $9.78 \pm 0.14$  RMSE on Los-loop PH1; consumed per timestep by T-GCN-MultiGSL, the same artifacts yield  $4.84 \pm 0.11$ . The union retains 28 of the 30 lagged edge slots, so the bulk of the edge content is present in both conditions. The opposite outcomes therefore *cannot be explained by the edge set alone*. Because GCN and T-GCN also differ in backbone (non-recurrent versus recurrent), this comparison is not a fully controlled isolation of consumption; it provides evidence that graph

**Table 5:** Temporal-resolution variant of Los-loop at 15-minute sampling (Sect. 4.9). PH  $k$  denotes 15 $k$  minutes ahead and is *not* comparable to PH  $k$  of the 5-minute series. Five seeds.

| Method                | PH1 (15 min)         | PH2 (30 min)         | PH3 (45 min)         | PH4 (60 min)         |
|-----------------------|----------------------|----------------------|----------------------|----------------------|
| T-GCN-NoSpatial       | 8.600 (0.249)        | 9.311 (0.197)        | 10.048 (0.074)       | 10.460 (0.196)       |
| T-GCN-MultiGSL        | 7.246 (0.298)        | 8.571 (0.327)        | 9.112 (0.077)        | 9.756 (0.210)        |
| T-GCN-MultiGSL-Mix    | <b>6.240 (0.187)</b> | <b>7.322 (0.338)</b> | <b>8.063 (0.203)</b> | <b>8.777 (0.326)</b> |
| Mix vs NoSpatial      | −27.4%               | −21.4%               | −19.8%               | −16.1%               |
| Mix wins (of 5 seeds) | 5                    | 5                    | 5                    | 5                    |

consumption is a major determinant of whether the learned multi-lag structure is useful.

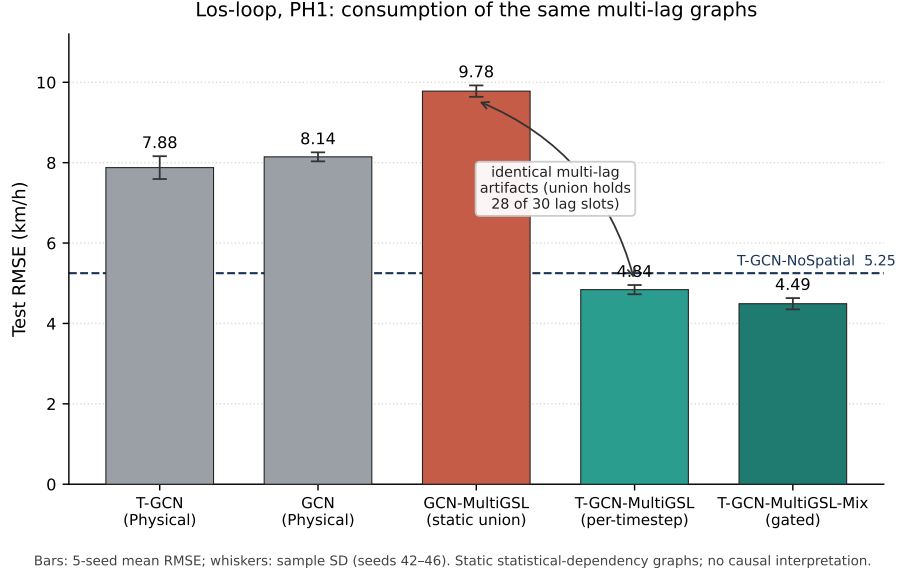
Within the T-GCN family, the consumption ladder is equally informative. Fixed cyclic lag assignment already beats NoSpatial on Los-loop (7.8% relative RMSE reduction at PH1). Adding three global scalars (Weighted) changes almost nothing (differences  $\leq 0.01$  RMSE), so a single static mixture over lags is not what drives the gain. Replacing that global mixture with a per-node, per-timestep gate (Mix) adds a further 0.35–0.40 RMSE on Los-loop in 5/5 seeds at every horizon (Sect. 5.5), without modifying the learned edge sets. The results thus indicate that the utility of the learned multi-lag structure depends critically on how the graphs are consumed: lag-aligned use helps where the graphs are informative, and finer selection helps further. The gate varies *use*, not *structure*; the graphs remain static fitted artifacts.

The cGSL–GSL contrast is a miniature of the same theme. Binary symmetrization of the same contemporaneous artifact helps the non-recurrent GCN substantially (cGSL 5.76 vs GSL 7.83 on Los-loop PH1) and is nearly irrelevant for T-GCN (5.82 vs 5.86). One consistent explanation is *aggregation compatibility*: a symmetric aggregator is better matched to a symmetric operator, whereas the recurrent cell can absorb directed asymmetry without the same penalty. We offer this as an observation of the current study, not as a general theorem about DAG versus cyclic graphs, and we do not revive a temporal reading of the contemporaneous DAG.

### 6.3 What the Learned Graph Represents

We describe the learned graphs as descriptive statistical dependency structure under a linear DAGMA fit with an acyclicity regularizer. The multi-lag construction stacks explicit lag blocks, and the consumption ablations show that treating those blocks as lag-specific operators is more effective than merging them into one static adjacency. That pattern is *consistent with* a lag-structured reading of the fitted matrices, but we did not independently validate the lag interpretation against ground-truth propagation, and acyclicity is an optimization constraint rather than causal evidence.

The dataset boundary sharpens this reading. On SZ-Taxi the multi-lag fit retains only two lagged edges, and every learned variant remains within seed variability of



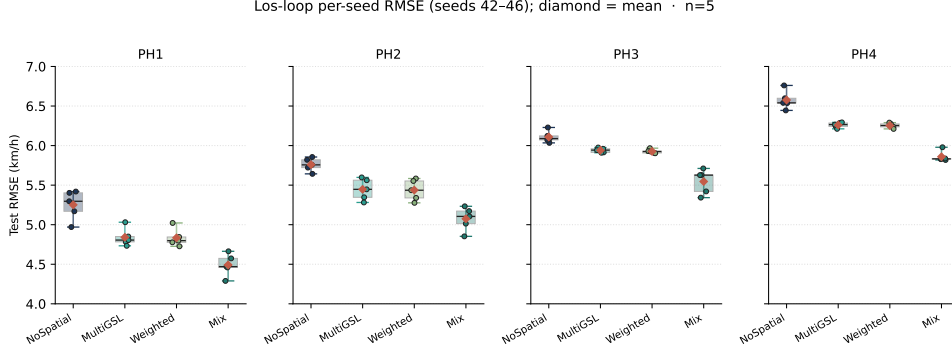
**Fig. 3:** Los-loop PH1: the same multi-lag DAGMA artifacts under static-union consumption (GCN-MultiGSL) versus per-timestep consumption (T-GCN-MultiGSL), with physical and gated references. Bars are five-seed mean RMSE; whiskers are sample standard deviations (seeds 42–46). The union retains 28 of the 30 lagged edge slots. Graphs are static statistical-dependency estimates; the figure does not encode causal influence.

the graph-free control (Sect. 5.6). On Los-loop, where the lag blocks are substantially denser (12/3/15 slots; union 28), lag-aligned consumption yields the study’s positive result, and the 15-minute resolution variant preserves a large relative improvement over NoSpatial at 15–60 minutes ahead (Table 5). The honest synthesis is conditional: learned multi-lag graphs help forecasting when the fitted dependency structure is informative enough to be worth consuming, and when the consumption mechanism respects the lag organization of that structure. Where the fit is nearly empty, the method collapses to the graph-free baseline.

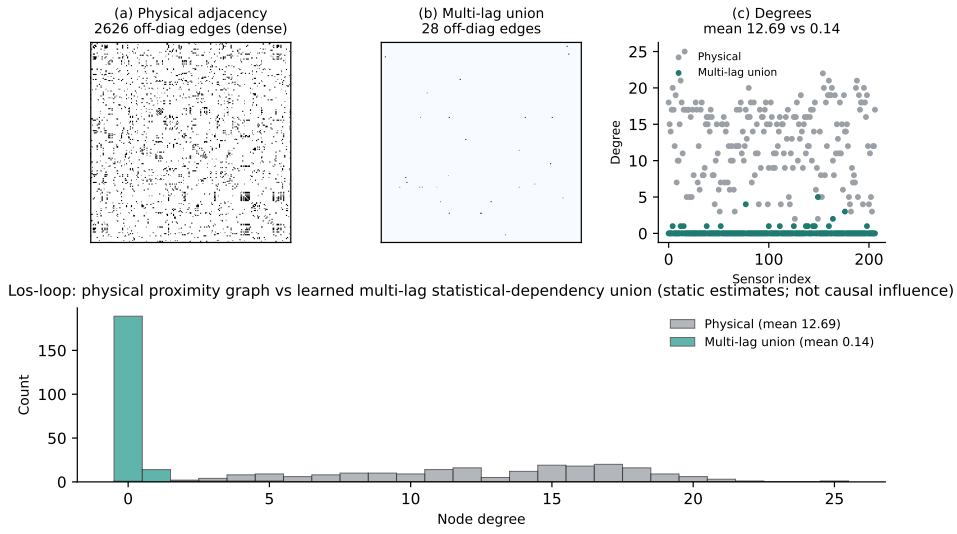
None of this implies that the physical graph is useless in absolute terms, or that learned graphs will help every network. It implies that the default practice of treating a dense proximity graph as ground truth, and of crediting any learned graph that improves on it, is incomplete without a graph-free control and without an explicit account of how the graph is consumed over time.

## 7 Limitations

The study is a controlled evaluation on two public benchmarks, not a claim of universal benefit. Several limitations bound the evidence.



**Fig. 4:** Los-loop per-seed test RMSE for the multi-lag consumption ladder (seeds 42–46). Diamonds mark five-seed means matching Table 2.  $n = 5$ ; see Sect. 4.7.



**Fig. 5:** Los-loop graph structure: (a) physical road-network adjacency (2626 off-diagonal edges; mean degree 12.69); (b) union of the three multi-lag DAGMA graphs after consumer thresholding (28 edges; mean degree 0.14); (c) and histogram of node degrees. Learned graphs describe statistical dependency structure, not causal traffic influence.

### *Statistical power.*

All main results aggregate five training seeds. Sample standard deviations and paired win counts are reported, and paired  $t$ -tests are given for reference, but with  $n = 5$  the exact Wilcoxon signed-rank test cannot fall below  $p = 0.0625$ . We therefore avoid definitive significance claims and rely on effect size, seed consistency, and the matched controls.

### ***Scope of the controls.***

The matched-sparsity and capacity controls are restricted to Los-loop at PH1. There is no matched-sparsity control on SZ-Taxi or at longer horizons, no sparsified variant of the physical graph, and no sensitivity sweep over DAGMA  $\lambda_1$  or threshold parameters. The sparsity conclusion is explicitly scoped to that single cell.

### ***Datasets and horizons.***

Only two datasets are used, with different sampling intervals (5 min and 15 min). Horizons are limited to  $\text{PH} \leq 4$  at the native sampling rate of each series. The 15-minute Los-loop experiment covers 15–60 minutes ahead as a temporal-resolution variant; it is not evidence for PH5–8 at 5-minute sampling, and PH indices are not comparable across sampling intervals.

### ***Graph-learning assumptions.***

Graphs are estimated with linear DAGMA ( $\ell_2$  score, acyclicity regularizer). Nonlinear dependency, measurement noise beyond the linear SEM assumption, and misspecification of the lag depth  $L = 3$  are not examined. Measured runtimes are non-negligible for the multi-lag fit (on the order of hours for the 828-variable construction on Los-loop), which limits rapid replication at larger  $N$ .

### ***Static graphs; gated use.***

The learned graphs are fitted once and remain static. The Mix mechanism varies how those static graphs are consumed per node and timestep; it does not produce time-varying edge sets. Genuinely time-varying learned graphs remain future work.

### ***Backbone scope.***

Experiments cover the GCN and T-GCN family used here. Other spatio-temporal architectures (attention-based models, diffusion variants, large transformers) are outside the study.

## **8 Conclusion**

We asked when and how learned graph structure helps graph-based traffic forecasting, separating the *source* of the graph from the *mechanism that consumes it*. Across two public benchmarks, twelve configurations, four horizons, and five seeds, three conclusions follow. First, the dense physical road-network graph is not a sound default: a graph-free identity control outperforms it in every dataset–horizon cell, and single learned contemporaneous graphs improve on the physical graph but never on the graph-free control. Second, lag-specific multi-lag DAGMA graphs are useful on Los-loop only when consumed in a temporally aligned manner; feeding the identical graphs as a static union is markedly worse, so consumption is at least as important as learning, while a global lag mixture is insufficient and a per-node, per-timestep gate adds a further seed-consistent gain (up to 14.5% relative RMSE reduction versus the graph-free baseline at PH1). Third, the benefit is conditional: on SZ-Taxi, where the learned



multi-lag graphs retain almost no edges, all learned variants remain within seed variability of the graph-free baseline, and matched-budget controls show that sparsity alone does not explain the Los-loop gain.

Future work includes longer horizons at 5-minute resolution, additional networks, sparsified-physical controls,  $\lambda$ /threshold sensitivity, other backbones, and genuinely time-varying graph structure. Practically, we recommend that graph-based traffic forecasters report a graph-free control and state explicitly how any learned graph is consumed over time.

## References

- [1] Akhtar, M., Moridpour, S.: A review of traffic congestion prediction using artificial intelligence. *Journal of Advanced Transportation* **2021** (2021)
- [2] Ye, J., Zhao, J., Ye, K., Xu, C.: How to build a graph-based deep learning architecture in traffic domain: A survey. *IEEE Transactions on Intelligent Transportation Systems* (2020)
- [3] Xie, P., Li, T., Liu, J., Du, S., Yang, X., Zhang, J.: Urban flow prediction from spatiotemporal data using machine learning: A survey. *Information Fusion* **59**, 1–12 (2020)
- [4] Chen, F., Zhang, Y., Chen, L., Meng, X., Qi, Y., Wang, J.: Dynamic traveling time forecasting based on spatial-temporal graph convolutional networks. *Front. Comput. Sci.* **17**(6) (2023) <https://doi.org/10.1007/s11704-023-2704-x>
- [5] Zhou, Z., Basker, R., Yeung, D.-Y.: Graph neural networks for multivariate time-series forecasting via learning hierarchical spatiotemporal dependencies. *Engineering Applications of Artificial Intelligence* **147**, 110304 (2025) <https://doi.org/10.1016/j.engappai.2025.110304>
- [6] Zhao, L., Song, Y., Zhang, C., Liu, Y., Wang, P., Lin, T., Deng, M., Li, H.: T-GCN: A temporal graph convolutional network for traffic prediction. *IEEE Trans. on Intell. Transportation Systems* **21**(9), 3848–3858 (2020)
- [7] Bai, J., Zhu, J., Song, Y., Zhao, L., Hou, Z., Du, R., Li, H.: A3t-gcn: Attention temporal graph convolutional network for traffic forecasting. *ISPRS International Journal of Geo-Information* **10**(7) (2021) <https://doi.org/10.3390/ijgi10070485>
- [8] Sun, C., Li, C., Lin, X., Zheng, T., Meng, F., Rui, X., Wang, Z.: Attention-based graph neural networks: a survey. *Artificial Intelligence Review* **56**(2), 2263–2310 (2023) <https://doi.org/10.1007/s10462-023-10577-2>
- [9] Chen, G., Chen, L., Li, Z., Wei, Y., He, Z., Yang, F.: Ta-stgcn: trend-aware attention spatio-temporal graph convolutional network for traffic flow prediction. *International Journal of Data Science and Analytics* **22**(1), 56 (2026) <https://doi.org/10.1007/s41060-026-01040-w>

- [10] Zheng, X., Aragam, B., Ravikumar, P., Xing, E.P.: Dags with no tears: Continuous optimization for structure learning. In: NeurIPS (2018)
- [11] Bello, K., Aragam, B., Ravikumar, P.: Dagma: learning dags via m-matrices and a log-determinant acyclicity characterization. In: Proceedings of the 36th International Conference on Neural Information Processing Systems. NIPS '22. Curran Associates Inc., Red Hook, NY, USA (2024)
- [12] Hamed, M.M., Al-Masaeid, H.R., Said, Z.M.B.: Short-term prediction of traffic volume in urban arterials. *Journal of Transportation Engineering* **121**(3), 249–254 (1995)
- [13] Fabian, X., Ban, G., Boussaïd, R., *et al.*: Modeling and forecasting vehicular traffic flow as a seasonal arima process. *Journal of Transportation Engineering* **129**(6), 664–672 (2003)
- [14] Smola, A.J., Schölkopf, B.: A tutorial on support vector regression. *Statistics and Computing* **14**(3), 199–222 (2004)
- [15] Lippi, M., Bertini, M., Frasconi, P.: Short-term traffic flow forecasting. *IEEE Transactions on Intelligent Transportation Systems* **14**(2), 871–882 (2013)
- [16] Ojeda, L.L., Kibangou, A.Y., Wit, C.C.: Adaptive kalman filtering for multi-step ahead traffic flow prediction. In: American Control Conference (2013)
- [17] Park, D., Rilett, L.R.: Forecasting freeway link travel times with a multilayer feed-forward neural network. *Computer-Aided Civil and Infrastructure Engineering* **14**(5), 357–367 (2010)
- [18] Zhang, J., Zheng, Y., Qi, D.: Deep spatio-temporal residual networks for citywide crowd flows prediction, 1655–1661 (2017)
- [19] Cao, X., Zhong, Y., Zhou, Y., Wang, J., Zhu, C., Zhang, W.: Interactive temporal recurrent convolution network for traffic prediction. *IEEE Access* **5**, 9893–9901 (2017)
- [20] Fu, R., Zhang, Z., Li, L.: Using lstm and gru neural network methods for traffic flow prediction. In: Youth Academic Conference of Chinese Association of Automation, pp. 324–328 (2017)
- [21] Khan, H.M., Khan, A., Villar, S.G., Lopez, L.A.D., Almaleh, A., Al-Qahtani, A.M.: A comparative study of optimized-lstm models using tree-structured parzen estimator for traffic flow forecasting in intelligent transportation. *Computers, Materials and Continua* **83**(2), 3369–3388 (2025) <https://doi.org/10.32604/cmc.2025.060474>
- [22] Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional

- networks. In: ICLR (2017)
- [23] Zuo, J., Zeitouni, K., Taher, Y., Garcia-Rodriguez, S.: Graph convolutional networks for traffic forecasting with missing values. *Data Mining and Knowledge Discovery* **37**(2), 913–947 (2023) <https://doi.org/10.1007/s10618-022-00903-7>
  - [24] Wang, B., Long, Z., Sheng, J., Zhong, Q.: Spatial-temporal similarity fusion graph adversarial convolutional networks for traffic flow forecasting. *Journal of the Franklin Institute* **361**(17), 107299 (2024) <https://doi.org/10.1016/j.jfranklin.2024.107299>
  - [25] Xia, Z., Zhang, Y., Yang, J., Xie, L.: Dynamic spatial-temporal graph convolutional recurrent networks for traffic flow forecasting. *Expert Systems with Applications* **240**, 122381 (2024) <https://doi.org/10.1016/j.eswa.2023.122381>
  - [26] Remmouche, B., Boukraa, D., Zakharova, A., Bouwmans, T., Taffar, M.: Long-term spatio-temporal graph attention network for traffic forecasting. *Expert Systems with Applications* **288**, 128244 (2025) <https://doi.org/10.1016/j.eswa.2025.128244>
  - [27] Zeb, A., Zheng, J., Ye, Y., Chen, J., Zhang, S., Wei, X., Jianqiao Yu, J.: Traffic forecasting with meta attentive graph convolutional recurrent network. *Expert Systems with Applications* **287**, 128073 (2025) <https://doi.org/10.1016/j.eswa.2025.128073>
  - [28] Jia, C., Jiang, F., Chen, Z., Wang, R., Xiao, Y.: A dynamic traffic flow prediction method based on spatio-temporal graph transformer. *Applied Intelligence* **55**(16), 1048 (2025) <https://doi.org/10.1007/s10489-025-06895-3>
  - [29] Liu, R., Shin, S.-Y.: A review of traffic flow prediction methods in intelligent transportation system construction. *Applied Sciences* **15**(7) (2025) <https://doi.org/10.3390/app15073866>
  - [30] Peng, L., Liao, X., Li, T., Guo, X., Wang, X.: An overview based on the overall architecture of traffic forecasting. *Data Science and Engineering* **9**(3), 341–359 (2024) <https://doi.org/10.1007/s41019-024-00246-x>
  - [31] Battaglia, P.W., Hamrick, J.B., Bapst, V., Sanchez-Gonzalez, A., Zambaldi, V., Malinowski, M., Tacchetti, A., Raposo, D., Santoro, A., Faulkner, R., et al.: Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261* (2018)
  - [32] Liu, H., Wei, J., Xu, T.: Community detection based on community perspective and graph convolutional network. *Expert Systems with Applications* **231** (2023) <https://doi.org/10.1016/j.eswa.2023.120748>

- [33] Liping Deng, B.G., Zheng, W.: GCN-based weakly-supervised community detection with updated structure centres selection. *Connection Science* **36**(1), 2291995 (2024) <https://doi.org/10.1080/09540091.2023.2291995>
- [34] Amintoosi, M.: Overlapping clusters in cluster graph convolutional networks. *Journal of Algorithms and Computation* **53**(2), 33–45 (2021)
- [35] Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W.L., Leskovec, J.: Graph convolutional neural networks for web-scale recommender systems. In: *KDD* (2018)
- [36] Chen, Y., Wu, L.: In: Wu, L., Cui, P., Pei, J., Zhao, L. (eds.) *Graph Neural Networks: Graph Structure Learning*, pp. 297–321. Springer, Singapore (2022). [https://doi.org/10.1007/978-981-16-6054-2\\_14](https://doi.org/10.1007/978-981-16-6054-2_14) . [https://doi.org/10.1007/978-981-16-6054-2\\_14](https://doi.org/10.1007/978-981-16-6054-2_14)